ROS2 Architecture and Custom Communication Pipelines

Ved Parmar
Formula Manipal Driverless Taskphase
Manipal, India

Abstract*—*This report details the architectural foundations of the Robot Operating System 2 (ROS2) and demonstrates its application through custom node pipelines. It covers the ROS2 environment setup, foundational methodology, computation graph, communication protocols, workspace architecture, Quality of Service (QoS), and custom interface implementation.

Keywords*—*ROS2, colcon, computation graph, nodes, topics, services, turtlesim, rqt.

I. Environment Setup and Methodology

Development began by configuring the environment with Ubuntu 22.04 and installing the ROS2 Humble distribution using Debian packages. Before developing the custom solutions, a comprehensive methodology was followed to understand core ROS2 principles through official tutorials [1].

A. Command Line Interface (CLI) Tools

The initial learning phase involved using turtlesim and rqt to visualize system behavior. This included a foundational understanding of nodes, topics, services, parameters, and actions. Additionally, rqt_console was utilized to view system logs, and standard ROS2 commands were used for launching nodes [1].

B. Client Libraries and Build System

The second training phase focused on development tools, primarily using colcon to build packages. This involved creating a dedicated workspace and custom packages. The methodology culminated in writing simple publisher and subscriber nodes in Python and C++, alongside creating and implementing custom .msg and .srv interface files [1].

C. Essential CLI Commands

Throughout the methodology phase, several core terminal commands were utilized to navigate and introspect the ROS2 environment [1]:

source /opt/ros/humble/setup.bash: Sources the global ROS2 installation, allowing the terminal to recognize ROS2 commands.

ros2 run <package_name> <executable_name>: Launches a specific node from a compiled package.

ros2 pkg create --build-type ament_python <name>: Initializes a new ROS2 package with a Python build structure.

colcon build --symlink-install: Compiles the workspace. The symlink flag links Python source files directly to the install directory, allowing for code modifications without needing to rebuild.

ros2 node list / ros2 topic list: Outputs a list of all currently active nodes or topics on the computation graph.

rqt_graph: Launches a graphical interface to visualize the active nodes and how they are connected via topics.

II. ROS2 Computation Graph and Nodes

The ROS2 computation graph is the peer-to-peer network of communicating processes that make up a robotic system. At the core of this graph are nodes, which are individual executable programs designed to perform specific tasks, such as

reading a sensor or controlling a motor. Nodes exchange data across the graph to function as a cohesive unit [1].

III. Communication Interfaces and Parameters

Topics vs. Services: Topics use a publisher-subscriber architecture for continuous, unidirectional data broadcasting. They are ideal for continuous data streams like command velocities or live sensor feeds. Services function on a client-server model, requiring a discrete request and yielding a response. You would use a service rather than a topic for quick computations or changing a robot's settings where an immediate confirmation is required [1].

Actions: Actions are built for long-running, interruptible tasks, consisting of a goal, continuous feedback during execution, and a final result [1].

Parameters: Parameters are node-specific configuration variables that act like function inputs, allowing developers to change a node's behavior at runtime without modifying the source code [1].

Custom Interfaces: Custom .msg and .srv files allow developers to define tailored data structures when standard ROS2 messages do not fit the application's needs [1].

QoS Settings: Quality of Service (QoS) profiles dictate how data is handled over the network, adjusting parameters like reliability and history to prioritize either guaranteed delivery or low-latency best-effort transmission [1].

IV. ROS Python Implementation Outline

A standard ROS2 Python node follows a structured object-oriented boilerplate to ensure proper communication within the graph. The standard program utilizes the following core functions and components [1]:

Imports: Scripts import the rclpy library (the ROS2 Python client API) and the Node class to access foundational ROS2 functionalities. Custom message types (e.g., .msg files) are also imported here.

Node Class Initialization (__init__): The custom class inherits from the Node module. Within the constructor, super().__init__('node_name') assigns the node its official identifier. This is also where publishers, subscribers, and timers are instantiated.

Publishers: A publisher is created using the create_publisher() function, which specifies the message type, the topic name, and the queue size to broadcast data out to the computation graph.

Subscriptions (Listeners): A listener or subscriber is established using the create_subscription() function. It actively monitors a specific topic and triggers a predefined callback function whenever a new message is received from that topic.

Timers: Created using the create_timer() function, a timer executes a specific callback function at regular, predetermined intervals (e.g., every 0.5 seconds). This allows a node to perform recurring tasks without pausing or blocking the rest of the node's operations.

Callbacks: These are custom functions triggered by specific events, such as a timer expiring or a subscriber receiving a new message. Callbacks handle the primary logic and data processing of the node.

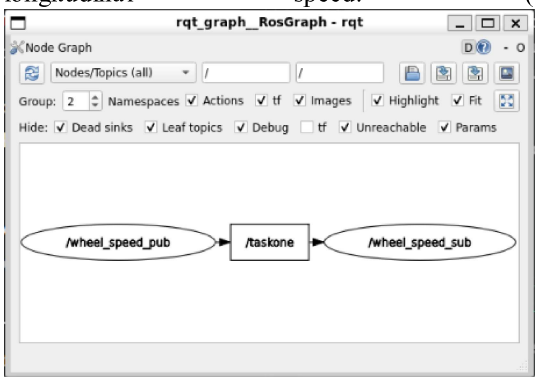
Main Function (def main()): This function uses rclpy.init() to initialize the ROS2 communication pipeline, instantiates the custom node class, and then calls rclpy.spin(node) to pause the execution thread and keep the node alive indefinitely. Finally, rclpy.shutdown() safely terminates the node when the process ends.

V. Solution Implementations

A. Task 1 (Wheel Speed)

The wheel_speed_pub node accepts terminal input and publishes a custom TaskOne message containing angular velocity and radius to the /taskone topic. The wheel_speed_sub node

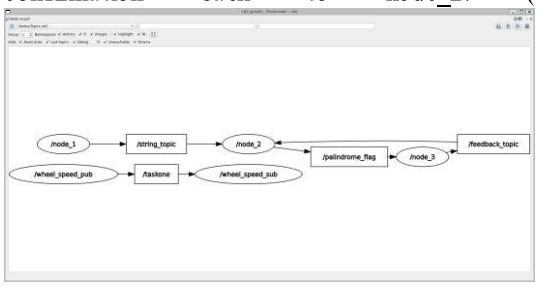
subscribes to this topic and calculates the longitudinal speed. (



)

B. Task 2 (Palindrome Pipeline)

This system uses a three-node pipeline. node_1 publishes a custom RT2String message to a primary topic. node_2 receives this string, calculates whether it is a palindrome, and publishes a binary flag to node_3. node_3 processes the flag and routes a "Yes" or "No" confirmation back to node_2. (



).

References

[1] Robotics Back-End, "ROS2 Tutorial - ROS2 Humble 2H50 [Crash Course]," YouTube. Available: https://www.youtube.com/watch?v=Gg25GfA456o